Automating Zygo Instruments with MetroScipt

Introduction

For Zygo instruments running with MetroPro software, MetroScript can be used to automate multiple and specialized operations with the ease of the click of a button. MetroScript provides a host of functionalities,

1. Change control values and settings;
2. Prompt the user for input enabling dynamic dialog;
3. Automatically calculate and save the results to files;
4. Shell the external Windows applications with expanded functionalities;
5. Allow the one-click batch process for repeated multiple actions.

In short, MetroScript is a very powerful script language that enables Zgyo users to write scripts to control complex measurement processes and to automate data flow. This is particular useful for instruments used on production floors.

While Zygo publishes a MetroScript programming language document, it is mostly a list of functions with limited explanation and few examples. This is in part due to fact that Zygo promotes its training class on the script language. But these training classes, starting at about $750/day, are very expensive propositions for many small companies.

Over the years, w2Lab has accumulated extensive experiences developing MetroScript for production automations and user interface improvements. By combining MetroScript with other Windows applications, we are able to do full automation by directly querying part tracking information from and sending measurement results to the manufacturing database.

The following are a few MetroScript examples demonstrating the powerful capability of the script language. The examples are only related to the generic data manipulations as we try to avoid the hardware specific discussions. However, it is important to point out that MetroScript has a rich set of functions enabling users to control all aspects of the instrument hardware (if available):

1. Interferometer control (acquire, analyze, lighting, and etc)
2. Stage motion controller (XYZ position and movement, auto-focus, and etc)
3. Fiducial locations (size, original, center, and etc)
4. Mask manipulation (size, location, and etc)

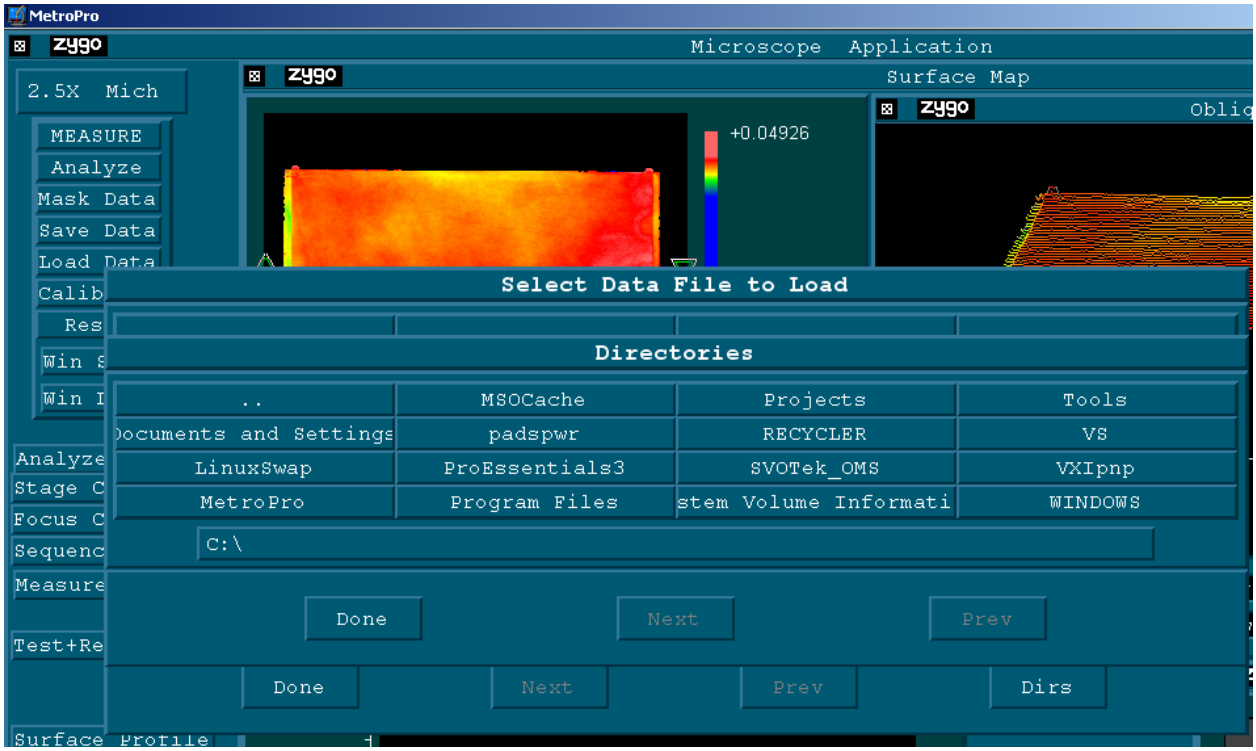
Actual scripts and associated Windows applications are accompanied with this document for trial. (Note that Windows applications are developed with Microsoft VS.Net and require the .NET framework being installed on the host computer. .NET framework is available for download from Microsoft web site.) Appendix A shows how to add MetroScript buttons to the MetroPro GUI linking to specific scripts. The scripts are executed by clicking these added MetroScript buttons.

 © 2006 w2Lab (www.w2Lab.com)

Example 1: Using Standard Windows Interface to Save and Load Data

One annoying feature of Zygo MetroPro is its awkward file browser associated with the Save Data and Load Data buttons (Figure 1A). This is especially true for users mostly used to the Microsoft Windows operation system. This example shows how to combine MetroScript and a Windows application to replace the file browser.

Figure 1A: Zygo MetroPro Standard File Browser



MetroScript has two data related functions called `savedata` and `loaddata` for, respectively, saving data to a file and loading data from a file. In addition, MetroScript has an `execute` function for shelling Windows applications. So, by building a simple Windows application for file browsing, one can build scripts to save and load data using standard Windows interface to achieve the same functionality as the build-in `Save Data` and `Load Data` buttons, but with the ease of familiar Windows interface.

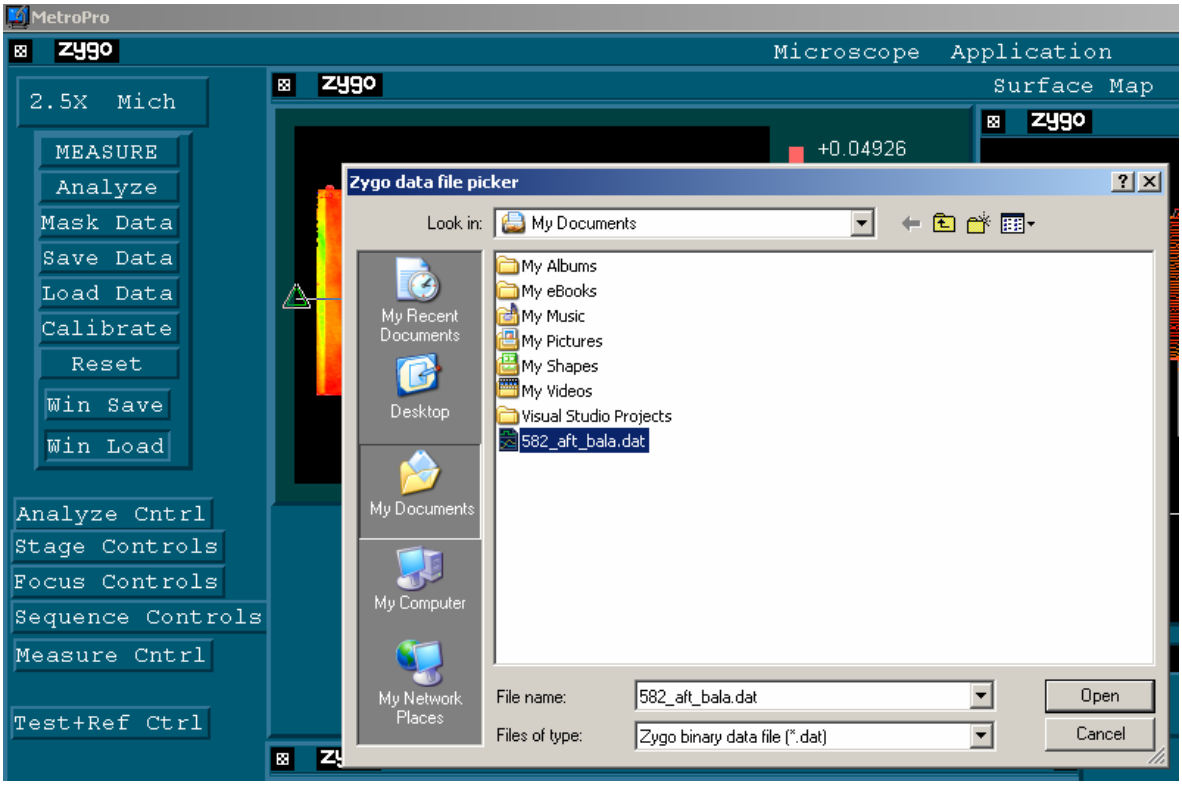
The scripts are made available on the Zygo GUI by adding two new MetroScipt buttons to the top left control button window by associating the script file name to the button (see Appendix A for the detail)

The following list is a simple 8-line script for loading data from files. First, it shells a Windows application called ZygoFileBrowser. Argument “1” indicates for open file browser which will do file existence checking and etc. Next, the script read the data exchange file to get the selected file name. Finally, if the file name is not an empty string, the script saves the current data to the file. The data exchanged file is used because the `execute` function does not allow sophisticated data exchange. The resulting application is shown in Figure 1B.

 © 2006 w2Lab (www.w2Lab.com)

```
! Zgyo Metro Script
! OpenFile.scr - This Script Calls ZygoFileBrowser
!                Windows application to browse and open a file
! © W2Laboratories/SVOTek
! Rev 001 04/14/2006

dim s$[1024]

!run file browser
ok = execute ("ZygoFileBrowser.exe 1")

!read file for file got picked
assign @f to "FileExchange.txt" "r"
enterline @f; s$

!close file
assign @f to ""

if s$ <> "" then
    loaddata(s$)
endif

!end
```

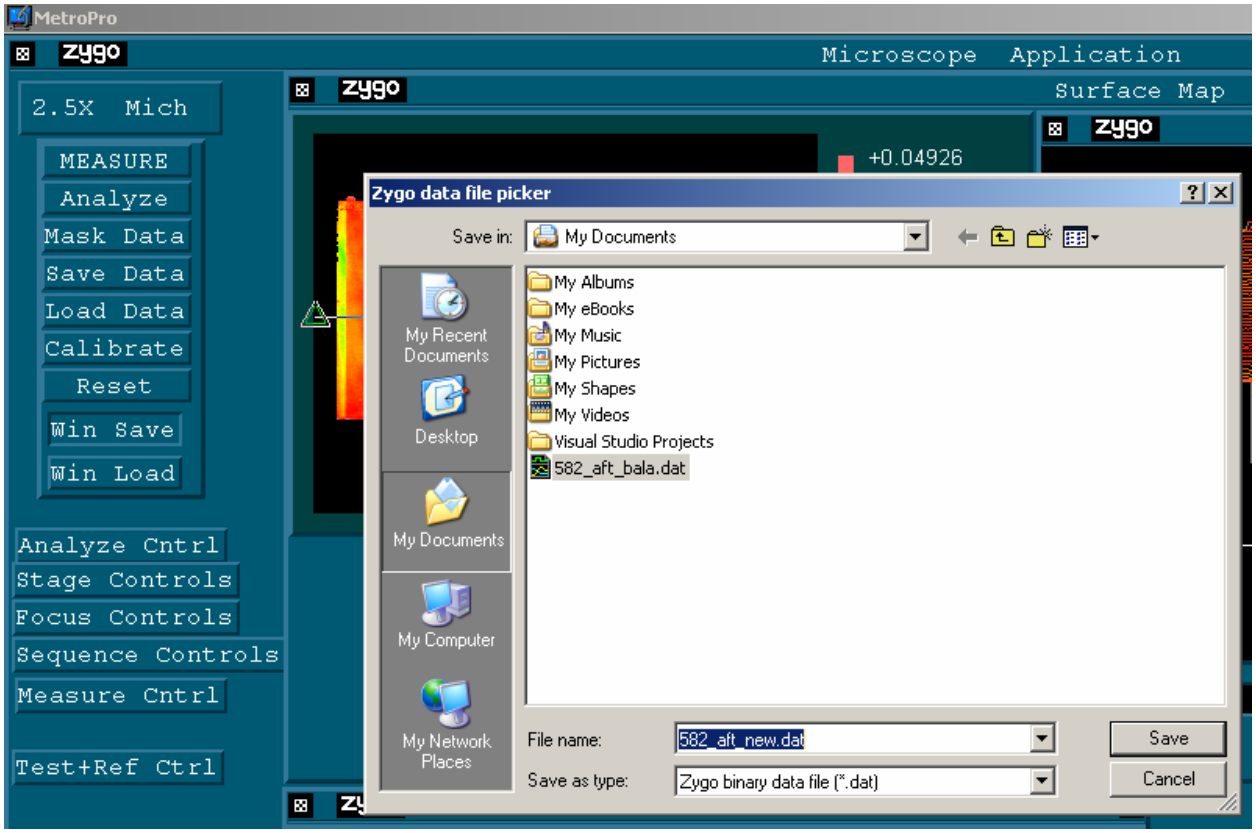
Figure 1B: Win Load User Interface for Loading Data



Similarly, one can build a script to save the data like the following:

```
! Zgyo Metro Script
! SaveFile.scr - This Script Calls ZygoFileBrowser
!                Window application to browse and save a file
! © W2Laboratories/SVOTek
! Rev 001 04/14/2006

dim s$[1024]

!run file browser
ok = execute ("ZygoFileBrowser.exe 2 ")

!read file for file got picked
assign @f to "FileExchange.txt" "r"
enterline @f; s$

!close file
assign @f to ""

if s$ <> "" then
    savedata (0, s$)
endif

!end
```

Figure 1C: Win Save User Interface for Saving Data



 © 2006 w2Lab (www.w2Lab.com)

Example 2: Saving Cross Section Data to a File

In many occasions, data such as surface profile plot need to be saved either for further analysis or for records. The following example show how to build a script to save the surface profile dynamically with user entered file names.

The script can be broke into 4 blocks. First, it opens a dialog box requesting user input of the file name and build a full file name using a preset data path (Figure 2A). Second, it gets the ID of data window (named as Cross Section) for which data is extracted. Third, it sets the parameter indicating how the data is saved (numeric data not image, file not printer, comma not tab delimited). And finally, call the `printwin` function to save the data. The last two lines are simple acknowledge telling user where the data is saved (Figure 2B).

Since the data is saved as comma delimited csv file, it can be easily open with Excel and have data processed and plotted using the Excel as shown in Figure 2C.

```
! Zgyo Metro Script
! CrossSection.scr - this script saves cross section profile to a file
! © W2Laboratories/SVOTek
! Rev 001 04/10/2006

dim s$[1024]
dim sFile$[100]

! Prompt user input for file name
s$ = " Enter a file name (maximum 10 letters) w/o file extension "
s$ = s$ & "\n script will add .csv as file extension"
SFile$= promptstr$(s$, "", 10)

! Build file name
sFile$ = "c:\\Tools\\" & SFile$& ".csv"

! Get window handler for the interested data
id = getwinid("Cross Section")

! Set parameters for saving data
!>> 0 = iamge, 1 = data
iData = 1
!>> 0 = printer, 1 = file
iFile = 1
!>> 1 = comma, 2 = tab
iFormat = 1

! Save data to the file
printwin(id, iData, iFile, iFormat, 0, 0, 1, sFile$)

! Acknowledge
s$ = "File " & sFile$ & " has been successfully saved!"
ok = dialog(s$, 1)

!end
```

 © 2006 w2Lab (www.w2Lab.com)

Figure 2A: Initial Dialog Requesting File Name

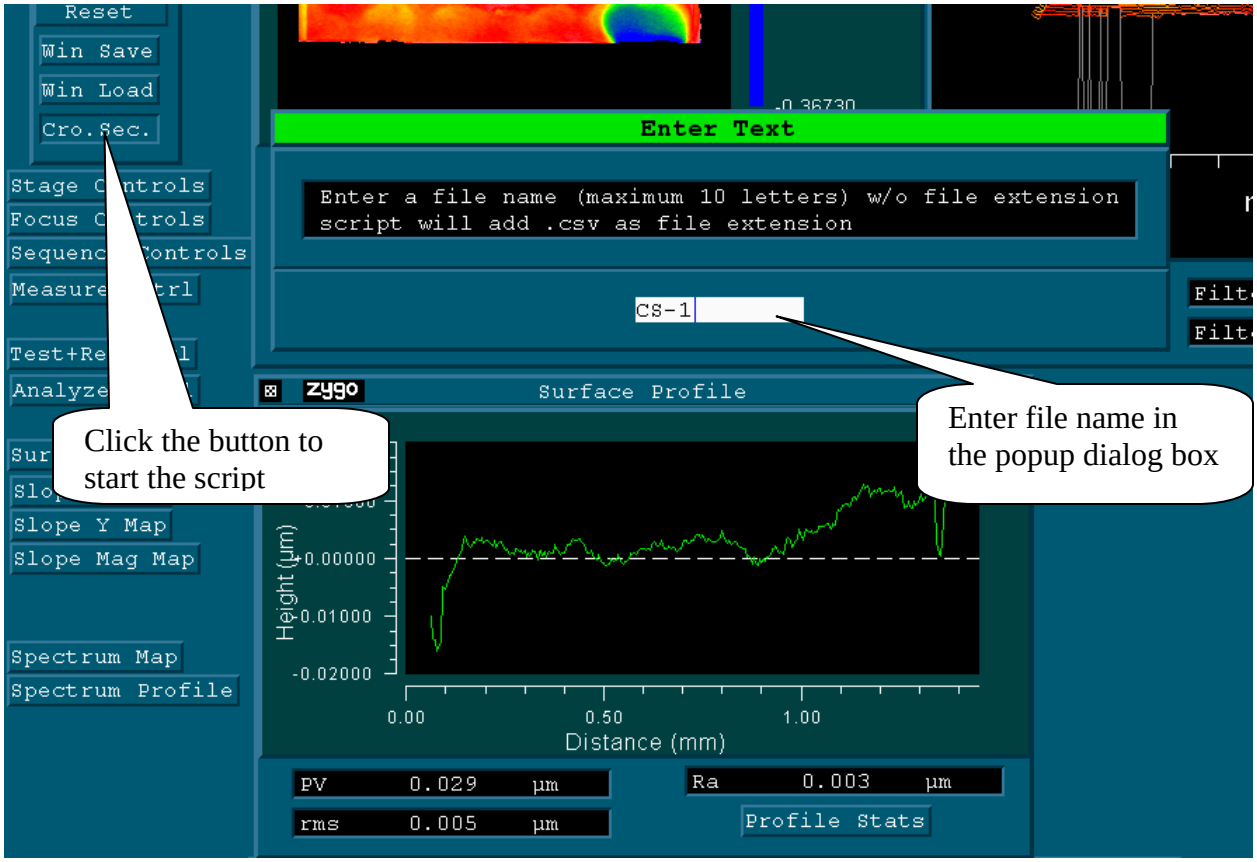
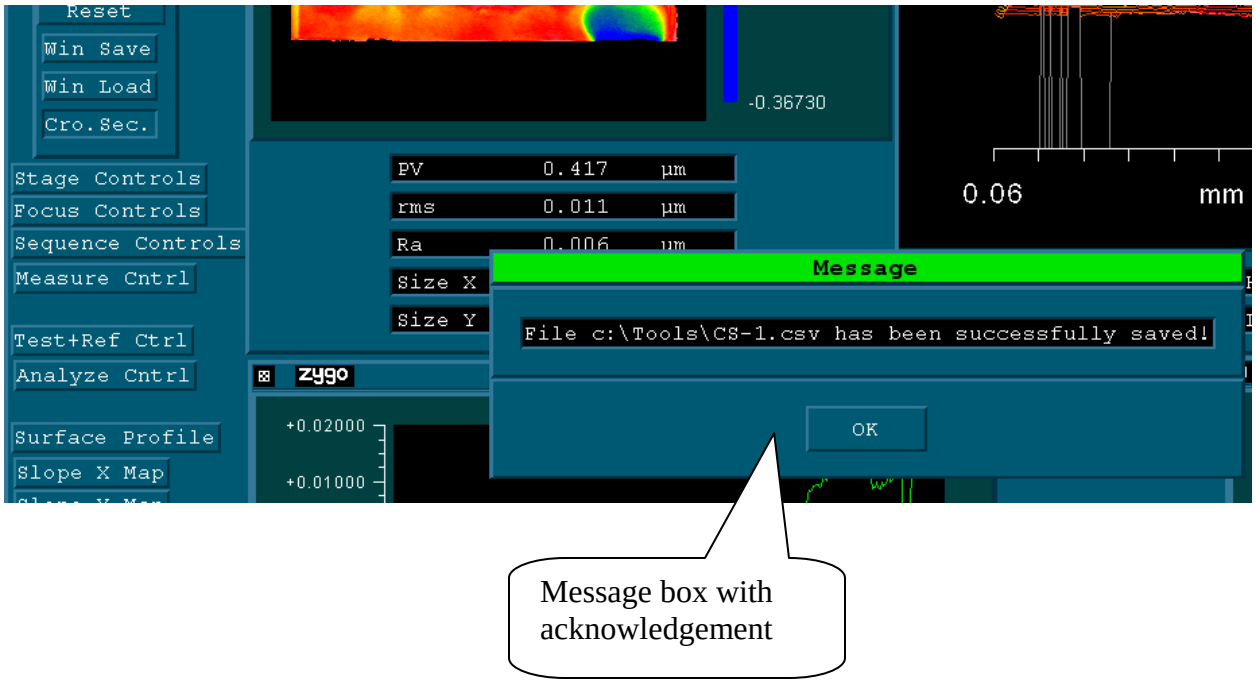
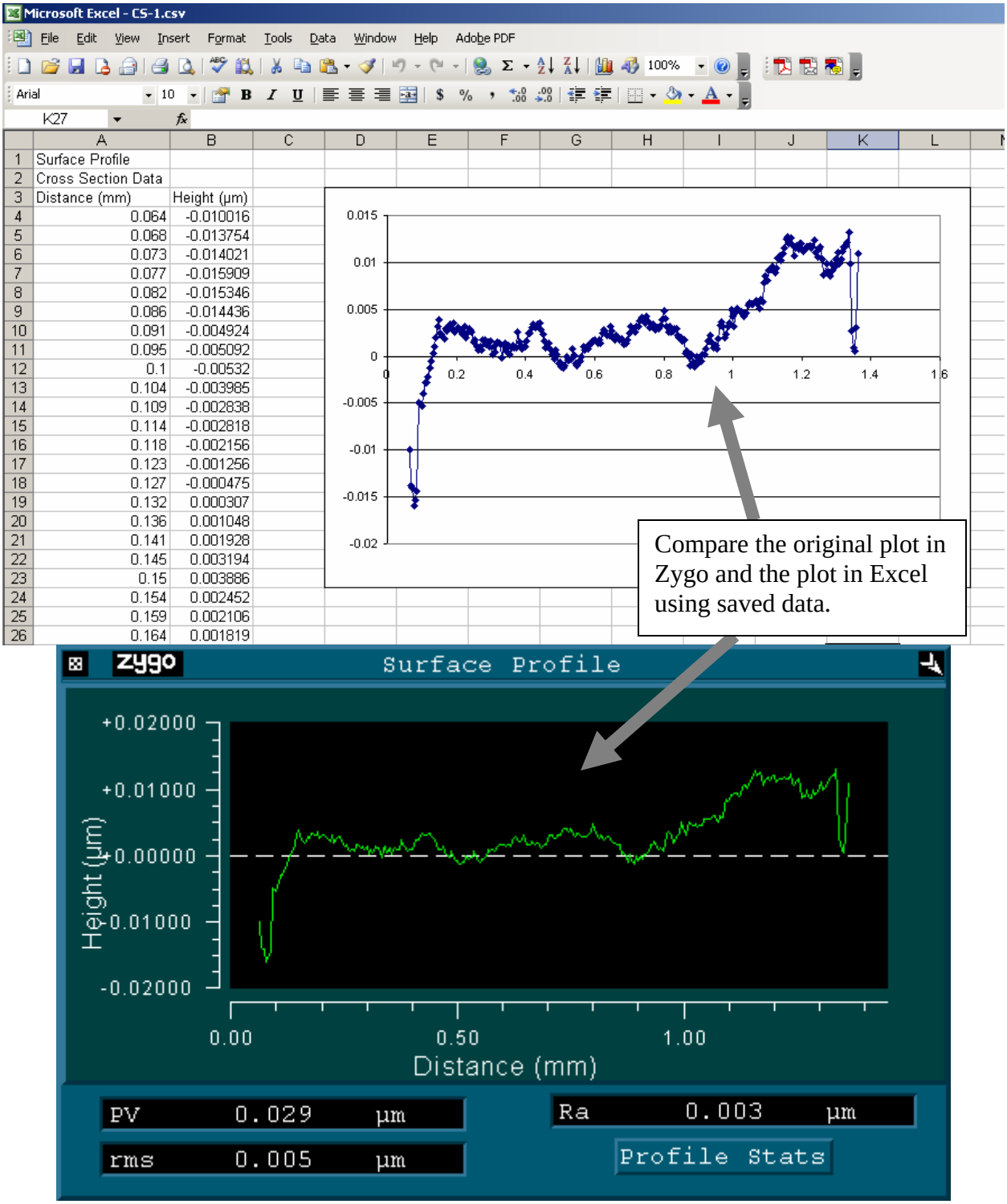


Figure 2B: Acknowledgement after Script Successfully Executed



 © 2006 w2Lab (www.w2Lab.com)

Figure 2C: Using Excel to Further Analyze the Saved Data



 © 2006 w2Lab (www.w2Lab.com)

Example 3: Save Measurement Results with Dynamic Dialogs

This example is an actual production script used to save measurement results to the production database. It is used in conjunction with a measurement script for a diode laser production line.

The script is rather long, but with extensive comments. It can be broke into four function blocks. At first, the script gets the part tracking information (serial number, process step name) and uses them to build file names for the data being saved. The way to get process name is through a numerated list, very much like what is done in old DOS programs. Next, the raw measurement data is saved to a file as binary data, and the surface map profile is saved as a bitmap file. The third block is to extract the measured summary data, such as size, peak-to-valley, root-mean-square, and etc, into a file. This block is rather long but very straightforward. Finally, an Windows application is called to move the data files (raw binary data, image, and summary data) to the network folder, and to read and parse the summary data file and download the summary data to the production database. In the end, an acknowledgement is shown indicating the successful end of the script.

Figures 3A-D show the progress of the script execution. Figure 3E show the results of the files.

```
! Zgyo Metro Script
! OPSScript.scr – OPS Data Saving Script
! © W2Laboratories/SVOTek
! Rev 011 02/22/2006

dim s$[1024]
dim sRoot$[100]
dim sFile$[100]

!---------------------------
! Get Serial Number
!---------------------------
id = getid("Controls/ Miscellaneous/ Part Ser Num")
promptentry(id)
sSN$ = getval$(id)

!---------------------------
! Reset Serial Number to up case
!---------------------------
sSN$ = upc$(sSN$)
setstr(id, sSN$)

!---------------------------
! Get Step Name by Number
!---------------------------
StepName:
s$ = "Enter a number (1-4) for the step: \n"
s$ = s$ & "1. Premount \n"
s$ = s$ & "2. Chip Itself \n"
s$ = s$ & "3. Post Process\n"
s$ = s$ & "4. Other \n"

sStep$ = promptstr$(s$, "", 1)
```

 © 2006 w2Lab (www.w2Lab.com)

```
!---------------------------
! Build step name
!----------------------------
if sStep$ = "1" then
    sStep$ = "Premount"
endif

if sStep$ = "2" then
    sStep$ = "Chip"
endif

if sStep$ = "3" then
    sStep$ = "Step3"
endif

if sStep$ = "4" then
    s$ = "Enter the step name (maximum 10 letters)"
    sStep$ = promptstr$(s$, "", 10)
endif

!---------------------------
! Validate and loop back if necessary
!----------------------------
if len(sStep$) = 1 then
    goto StepName
else

!---------------------------
! Build file name - root
!----------------------------
sRoot$ = "c:\\Tools\\" & sSN$ & "_" & sStep$

!---------------------------
! Save step name as comments
!---------------------------
id = getid("Controls/ Miscellaneous/ Comment")
setstr(id, sStep$)

!---------------------------
! Save data to file
!---------------------------
sFile$ = sRoot$ & ".dat"
savedata(0, sFile$)

!---------------------------
! Get screen print
!---------------------------
id = getwinid("Surface Map")
sFile$ = sRoot$ & ".bmp"
!>> 0 = iamge, 1 = data
iImage% = 0
!>> 0 = printer, 1 = file
iFile% = 1
!>> 0 = bmp file, 1 = tif file
iFormat% = 0
printwin(id, iImage%, iFile%, iFormat%, 0, 0, 0, sFile$)
```

 © 2006 w2Lab (www.w2Lab.com)

```
!---------------------------
! Get values and etc to a file
!---------------------------
sFile$ = sRoot$ & ".txt"
assign @f to sFile$ "w"

output @f; "[Summary]"

output @f; "Chip Number = " & sSN$
output @f; "Step Name   = " & sStep$
output @f; "File Root   = " & sRoot$
output @f; "Meas Date   = " & date$(timedate)
output @f; "Meas Time   = " & time$(timedate)

!-->> Peak/Valley
i = getid("Surface Wavefront Map/ Results/ Peak Valley/ PV")
v = getval(i, "um")
output @f; "P/V (um)    = " & val$(v)

!-->> size x
i = getid("Surface Wavefront Map/ Results/ Size/ Size X")
v = getval(i, "mm")
output @f; "Size X (mm) = " & val$(v)

!-->> size y
i = getid("Surface Wavefront Map/ Results/ Size/ Size Y")
v = getval(i, "mm")
output @f; "Size Y (mm) = " & val$(v)

!-->> Radius X
i = getid("Surface Wavefront Map/ Results/ RadCrv/ RadCrv X")
v = getval(i, "m")
output @f; "RadiusX (m) = " & val$(v)

!-->> Radius Y
i = getid("Surface Wavefront Map/ Results/ RadCrv/ RadCrv Y")
v = getval(i, "m")
output @f; "RadiusY (m) = " & val$(v)

!-->> Ra - map
i = getid("Surface Wavefront Map/ Results/ Ra")
v = getval(i, "nm")
output @f; "Map Ra (nm) = " & val$(v)

!-->> rms - map
i = getid("Surface Wavefront Map/ Results/ rms")
v = getval(i, "nm")
output @f; "Map rms (nm)= " & val$(v)

!------------------------------------------>> empty line
output @f; ""

!-->> Peak/Valley
i = getid("Surface Wavefront Profile/ Results/ Peak Valley/ PV")
v = getval(i, "um")
output @f; "Pro PV (um) = " & val$(v)
```

 © 2006 w2Lab (www.w2Lab.com)

```
!-->> radius
i = getid("Surface Wavefront Profile/ Results/ RadCrv")
v = getval(i, "m")
output @f; "Pro R (m)   = " & val$(v)

!-->> Ra - profile
i = getid("Surface Wavefront Profile/ Results/ Ra")
v = getval(i, "nm")
output @f; "Pro Ra (nm) = " & val$(v)

!-->> rms, profile
i = getid("Surface Wavefront Profile/ Results/ rms")
v = getval(i, "nm")
output @f; "Pro rms (nm)= " & val$(v)

!close file
assign @f to ""

!---------------------------
! shell Window Application to save data
!---------------------------
s$ = "DataToDB " & sFile$
ok = execute (s$)

!---------------------------
! finishing message
!---------------------------
ok = dialog(" Script finished successfully!", 1)

!end
```

Figure 3A: Popup Dialog Box Requesting Part Serial Number

 © 2006 w2Lab (www.w2Lab.com)

Figure 3B: Popup Dialog Box Requesting Process Step

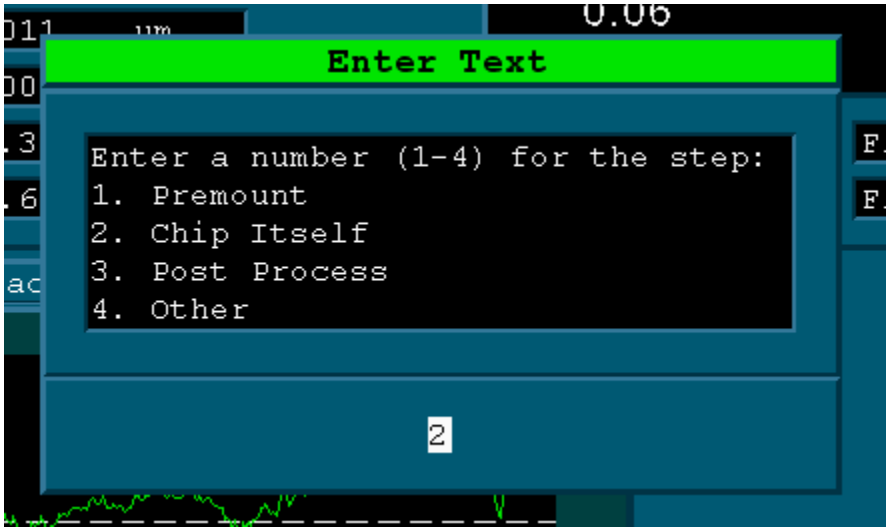


Figure 3C: Popup Dialog Box Requesting Process Step Name
(If "4" is selected in Figure 3B)

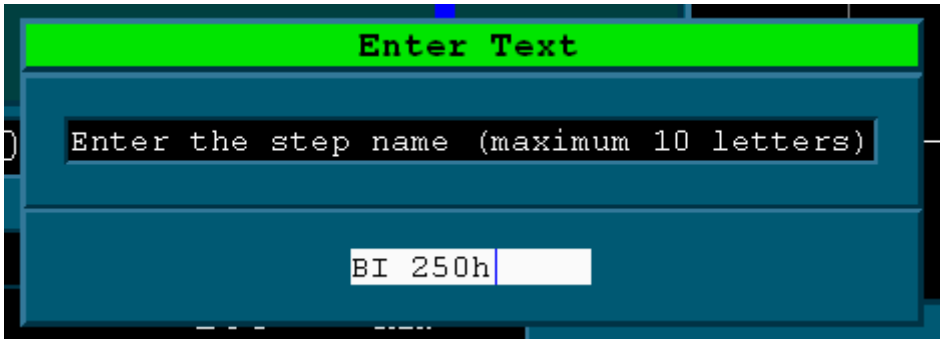
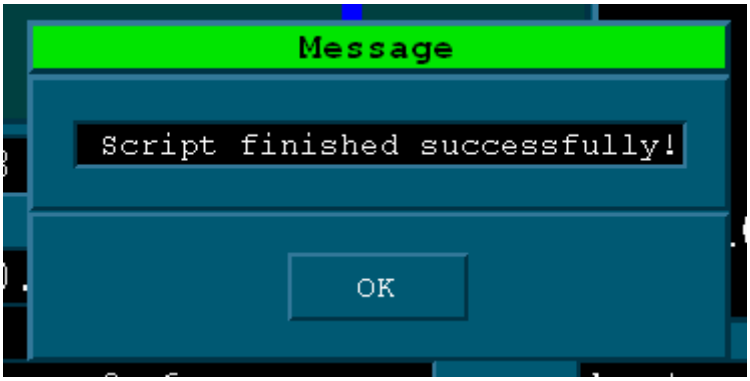
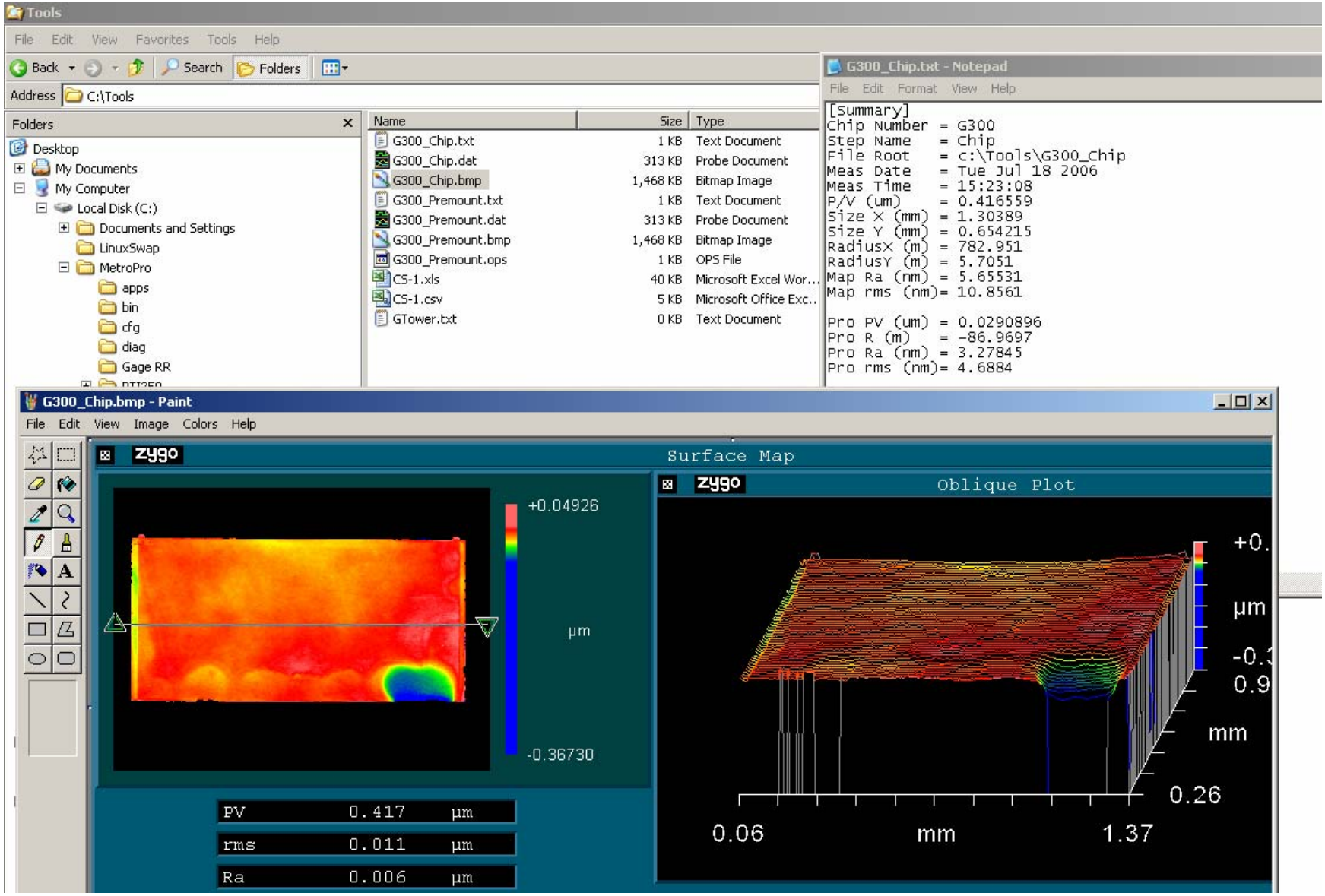


Figure 3D: Acknowledgement after Script Successfully Executed



 © 2006 w2Lab (www.w2Lab.com)

Figure 3E: Resulting File and Summary Data

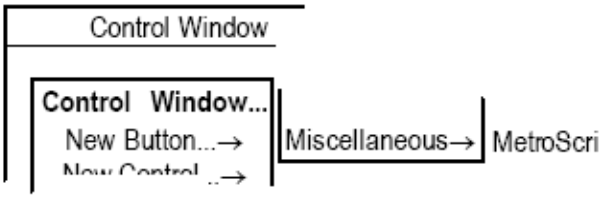


 © 2006 w2Lab (www.w2Lab.com)

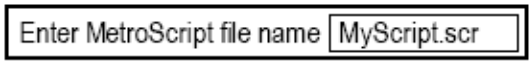
Appendix A: Adding MetroScript Button to MetroPro

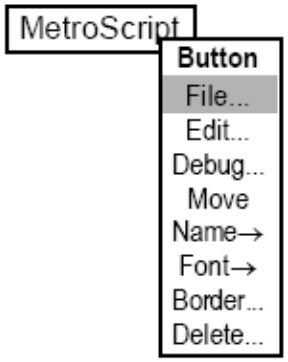
To access the created MetroScript file, one adds a MetroScript button to a MetroPro application and links it to a script file following 5 basic steps:

1. In MetroPro, create a MetroScript button in a Control window. Use the Control window menu and select New Button → Miscellaneous → MetroScript.



2. Specify the name of the script file to link to the MetroScript button. The file name should have ".scr" as the extension. Access the MetroScript button menu, select the File command, enter the script file name, and press [enter]

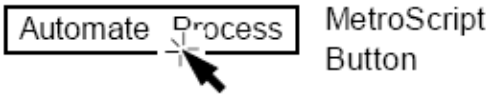




3. If the script has not been created, create a script file using any standard text editor. You can run a text editor from MetroPro by accessing the MetroScript button menu and selecting the Edit command. Save the file to the MetroPro working directory.

4. Change the name of the MetroScript button to indicate its function. Access the MetroScript button menu, select the Name→ Rename command, enter a new name, and press [Enter].

5. Click the button to run the script.



 © 2006 w2Lab (www.w2Lab.com)